MINI-FORMATION POUR DÉBUTANTS

Comment faire un commit sur Git ?

Fiche apprenant · 16 minutes · 3 modules

LA MÉTHODE EN UNE LIGNE

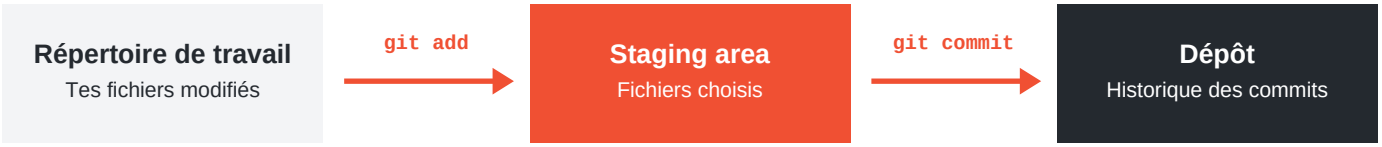
Vérifier avec `git status`, préparer avec `git add`, puis enregistrer avec `git commit -m`.

La formation en un coup d'œil

MODULE	TU VAS APPRENDRE À…	DURÉE
1. Vérifier les modifications	Identifier les fichiers modifiés et non suivis avec `git status`	4 min
2. Préparer les fichiers	Choisir les fichiers à inclure avec `git add`	5 min
3. Créer le commit	Enregistrer avec un message clair et vérifier l'historique	7 min

TOTAL : 16 MIN

Le trajet d'une modification



Mots clés

Dépôt (repository)	Dossier du projet dont Git suit l'historique.
Commit	Enregistrement de modifications dans l'historique du projet.
Zone de préparation (staging area)	Espace dans lequel les modifications sont sélectionnées avant leur enregistrement dans un commit.
Message de commit	Courte description du changement enregistré.
Fichier non suivi (untracked file)	Fichier que Git ne suit pas encore dans le dépôt.

MODULE 01

Module 1 · Vérifier les modifications

Objectif : identifier les fichiers modifiés et les fichiers non suivis avant de créer un commit.

1. Lancer la commande

1. Ouvre ton terminal dans le dossier de ton projet Git.
2. Tape `git status`.
3. Cette commande ne modifie rien : tu peux la relancer autant de fois que tu veux.

2. Lire le résultat

- `Changes not staged for commit` : fichiers suivis dont le contenu a changé, pas encore préparés.
- `Untracked files` : fichiers que Git ne suit pas encore.

3. Décider quoi inclure

Pour chaque fichier, demande-toi : **fait-il partie du changement que je veux enregistrer ?**

`index.html` (travail terminé) → **à inclure**. `Notes.txt` (brouillons) → **à laisser de côté**.

ILLUSTRATION

Illustration simplifiée : ton terminal peut afficher des lignes d'aide en plus.

À RETENIR

`git status` montre l'état du dépôt avant de décider quelles modifications préparer. Il ne crée pas de commit.

MODULE 02

Module 2 · Préparer les fichiers

Objectif : sélectionner les fichiers à inclure dans le prochain commit.

1. Préparer un fichier précis

1. Tape `git add index.html`.
2. La commande n'affiche rien et ne crée pas de commit : elle place le fichier dans la staging area.

2. Vérifier le résultat

1. Tape `git status`.
2. `index.html` apparaît dans `Changes to be committed`.
3. `Notes.txt` reste dans `Untracked files` : il est volontairement laissé de côté.

3. Deux façons d'utiliser git add

- `git add nom_du_fichier` prépare un fichier précis.
- `git add .` prépare les modifications du répertoire courant et de ses sous-répertoires.
- **Attention :** `git add .` peut inclure des fichiers que tu ne veux pas enregistrer (ici, `Notes.txt`). Vérifie toujours avec `git status`.

ILLUSTRATION

Schéma : seul le fichier choisi passe dans la staging area. Aucun commit n'est créé.

BON À SAVOIR

Fichier préparé par erreur ? Retire-le avec `git restore --staged nom_du_fichier`. Fichier modifié après `git add` ? Prépare-le à nouveau.

MODULE 03

Module 3 · Créer le commit

Objectif : enregistrer les modifications préparées dans l'historique, avec un message clair.

1. Créer le commit

1. Tape `git commit -m "Ajout du paragraphe de bienvenue"`.
2. L'option `-m` introduit le message du commit.

2. Vérifier

1. Tape `git log --oneline` : le nouveau commit apparaît en haut.
2. Tape `git status` : `Notes.txt` est toujours non suivi, comme voulu.

3. Écrire un bon message

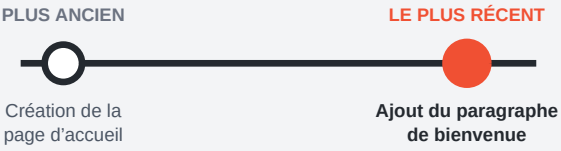
MESSAGE CLAIR	TROP VAGUE
Ajout du paragraphe de bienvenue	Modification du projet
Correction du lien d'accueil	test
Ajout du formulaire de contact	modif

4. Corriger le tir

CE QUE TU VOIS	ESSAIE CECI
`no changes added to commit`	Tu as oublié de préparer : lance `git add` d'abord.
Un brouillon est dans le commit	Retire-le avec `git restore --staged` avant de commiter.
Commande non reconnue	Vérifie la faute de frappe (`gti` au lieu de `git`).

5. Auto-vérification : mon premier commit

- ☐ J'ai lancé `git status` et regardé chaque fichier.
- ☐ J'ai préparé uniquement les fichiers voulus.
- ☐ J'ai créé le commit avec un message clair.
- ☐ J'ai vérifié l'historique avec `git log --oneline`.



L'historique de ton projet, commit après commit.